# Fiche de procédure de déploiement

Application Kampus : site statique GitHub Pages et backend Firebase

| | |
|---|---|
| **Dépôt GitHub** | github.com/GhostChen444/kampus |
| **Site en ligne** | https://ghostchen444.github.io/kampus/ |
| **Projet Firebase** | kampus-5c415 (Authentication, Firestore) |
| **Fichier principal** | index.html (interface, code et configuration en un seul fichier) |

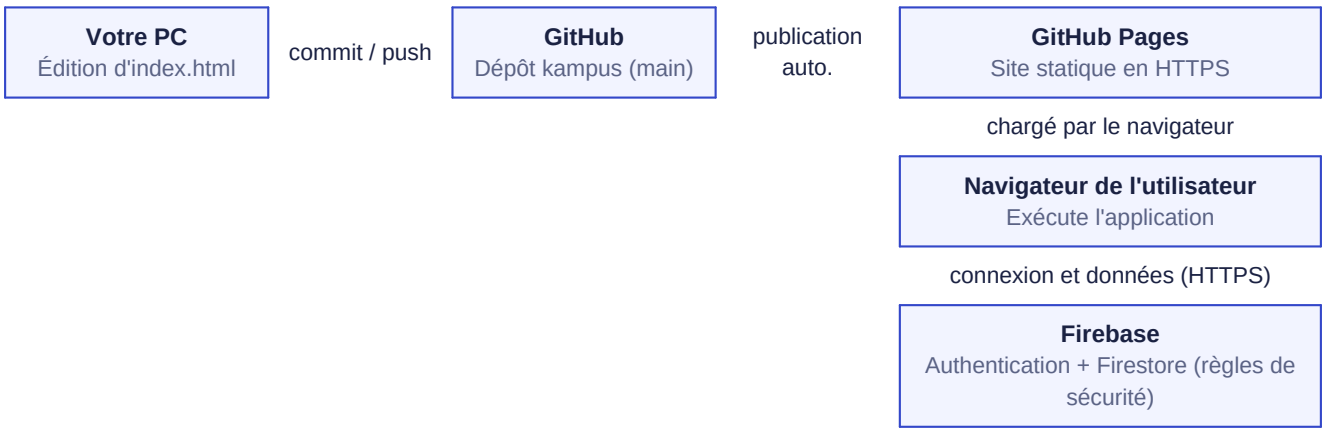
## 1. Comment s'appelle ce type de déploiement ?

Il s'agit du **déploiement d'une application web statique (SPA, Single Page Application) hébergée sur GitHub Pages, associée à un backend serverless de type BaaS (Backend as a Service), ici Firebase**. On rattache cette approche à l'architecture **Jamstack** : le front-end est servi sous forme de fichiers statiques, et les fonctions dynamiques (comptes utilisateurs, base de données) sont fournies par des services managés accessibles depuis le navigateur.

Les termes à retenir :

| Terme | Signification dans ce projet |
|---|---|
| Hébergement statique | GitHub Pages ne sert que des fichiers (ici index.html). Aucun serveur applicatif à gérer. |
| Serverless / BaaS | Firebase fournit l'authentification et la base Firestore. Il n'y a pas de serveur à installer ni à maintenir. |
| Déploiement continu (CD) | Chaque modification validée (commit) sur la branche main republie automatiquement le site. |
| Client-side rendering | Toute la logique s'exécute dans le navigateur, qui dialogue directement avec Firebase. |

## 2. Schéma de fonctionnement




## 3. Prérequis

Un compte GitHub, un compte Google (pour Firebase), un navigateur récent. Git n'est pas indispensable : le dépôt peut être alimenté par téléversement depuis l'interface GitHub.

## 4. Procédure : dépôt GitHub et GitHub Pages

| N° | Action | Vérification | OK |
|---|---|---|---|
| 1 | GitHub : **New repository**, nom **kampus**, visibilité Public (requise pour Pages gratuit), sans README ni licence. | Le dépôt s'affiche, vide. | |
| 2 | **Add file > Upload files** : déposer index.html (puis, si souhaité, firestore.rules et README.md). **Commit changes**. | Les fichiers apparaissent sur la branche main. | |
| 3 | **Settings** (Décors) > **Pages** > Source : *Deploy from a branch*, branche **main**, dossier **/ (root)**, **Save**. | Bandeau « Your site is live » avec l'adresse du site. | |
| 4 | Ouvrir https://ghostchen444.github.io/kampus/ après une à deux minutes. | L'écran de connexion de Kampus s'affiche. | |

## 5. Procédure : projet Firebase

| N° | Action | Vérification | OK |
|---|---|---|---|
| 1 | console.firebase.google.com > **Créer un projet** (nom kampus), Google Analytics désactivé. | Identifiant généré : kampus-5c415. | |
| 2 | **Authentication** > Commencer > Méthode de connexion > **E-mail/Mot de passe** > activer > Enregistrer. | Le fournisseur apparaît comme activé. | |
| 3 | Authentication > Utilisateurs > **Ajouter un utilisateur** (e-mail et mot de passe). | L'utilisateur est listé. | |
| 4 | **Firestore Database** > Créer une base (default), mode **Production**, région proche (Europe). | L'onglet Données est accessible. | |
| 5 | Firestore > onglet **Règles** : coller les règles ci-dessous puis **Publier**. | Historique des règles daté de l'heure de publication. | |
| 6 | Paramètres du projet > Vos applications > icône **Web** > enregistrer l'application > copier la configuration. | Bloc firebaseConfig affiché. | |
| 7 | Authentication > **Paramètres > Domaines autorisés** > ajouter **ghostchen444.github.io** (sans https:// ni /kampus). | Le domaine figure dans la liste. | |

Règles de sécurité Firestore (chaque utilisateur n'accède qu'à ses propres données) :

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {
    match /users/{uid}/{document=**} {
      allow read, write: if request.auth != null && request.auth.uid == uid;
    }
  }
}
```

## 6. Procédure : relier l'application à Firebase

| N° | Action | Vérification | OK |
|---|---|---|---|
| 1 | Dans index.html, bloc **1. CONFIGURATION FIREBASE** : renseigner apiKey, authDomain, projectId, storageBucket, messagingSenderId et appId. | Le projectId vaut kampus-5c415. | |
| 2 | Mettre **DEMO_ON = false** pour désactiver le compte de démonstration admin / admin. | La mention « Démo » disparaît de l'écran de connexion. | |
| 3 | Remplacer index.html sur GitHub (icône crayon ou Upload files) puis **Commit changes**. | Le site est republié automatiquement en une à deux minutes. | |

La clé apiKey d'une application web Firebase n'est pas un secret : elle identifie le projet. La sécurité des données repose sur les règles Firestore et sur l'authentification.

## 7. Tests de recette

| N° | Action | Vérification | OK |
|---|---|---|---|
| 1 | Se connecter avec l'e-mail et le mot de passe créés dans Firebase. | Accès à l'application, écran de dépôt de l'emploi du temps. | |
| 2 | Créer des matières (nom obligatoire, jour et horaires facultatifs). | Les rubriques apparaissent sur l'accueil. | |
| 3 | Dans une rubrique : ajouter une note, une capture avec description, un travail. | Les éléments s'affichent et persistent après rechargement. | |
| 4 | Firestore > Données : vérifier la collection **users**. | Un document par utilisateur, avec ses sous-collections. | |
| 5 | Tester sur téléphone (HTTPS) : appareil photo, navigation du bas, formulaires. | Aucun défilement horizontal, photo enregistrée. | |
| 6 | Se déconnecter puis se reconnecter depuis un autre appareil. | Les données sont retrouvées. | |

## 8. Mise à jour du site (procédure courante)

1. Modifier index.html. 2. Le remplacer sur GitHub et valider (Commit changes). 3. Attendre la republication automatique (une à deux minutes). 4. Recharger la page avec Ctrl+F5 pour contourner le cache du navigateur. Les données des utilisateurs, stockées dans Firebase, ne sont pas affectées par une mise à jour du site.

## 9. Dépannage

| Symptôme | Cause probable | Action |
|---|---|---|
| Connexion impossible sur le site en ligne | Domaine non autorisé | Ajouter ghostchen444.github.io dans Authentication > Paramètres > Domaines autorisés. |
| « E-mail ou mot de passe incorrect » | Compte absent ou saisie erronée | Vérifier l'utilisateur dans Authentication; saisir l'adresse e-mail complète. |
| « Accès refusé : vérifiez les règles Firestore » | Règles non publiées ou modifiées | Republier les règles de la section 5. |
| Page blanche ou ancienne version | Cache ou publication en cours | Attendre deux minutes, puis Ctrl+F5; vérifier l'onglet Actions du dépôt. |
| Erreur 404 sur l'adresse du site | Pages non activé ou mauvais dossier | Vérifier Settings > Pages (main, / root) et la présence d'index.html à la racine. |
| Caméra intégrée indisponible | Accès caméra refusé ou page non sécurisée | Autoriser la caméra, ou utiliser « Appareil photo du téléphone » ou la galerie. |

## 10. Points de vigilance

**Sécurité.** Ne jamais publier de règles Firestore ouvertes à tous (allow read, write: if true). Garder DEMO_ON à false en production. Activer un mot de passe robuste pour chaque compte.

**Stockage des images.** Les captures sont compressées puis enregistrées dans Firestore (limite d'environ 1 Mo par document). Pour un usage intensif, prévoir Firebase Storage.

**Quotas.** Le forfait gratuit Firebase (Spark) impose des quotas d'utilisation : les consulter dans la console avant d'ouvrir l'application à de nombreux utilisateurs.

**Sauvegarde.** Conserver sur GitHub le fichier firestore.rules et la documentation du projet, qui servent d'historique et de référence.